Deploying NGINX Using Different Base Images & Comparing Image Layers

This project demonstrates how to deploy **NGINX** containers using different base images and analyze how image size and layers differ between them.

We use Docker to run NGINX and inspect the image history to understand how lightweight images improve performance and efficiency.

Objective

- Deploy NGINX using different base images
- Compare Docker image layers
- Understand how base images affect image size and build complexity

---

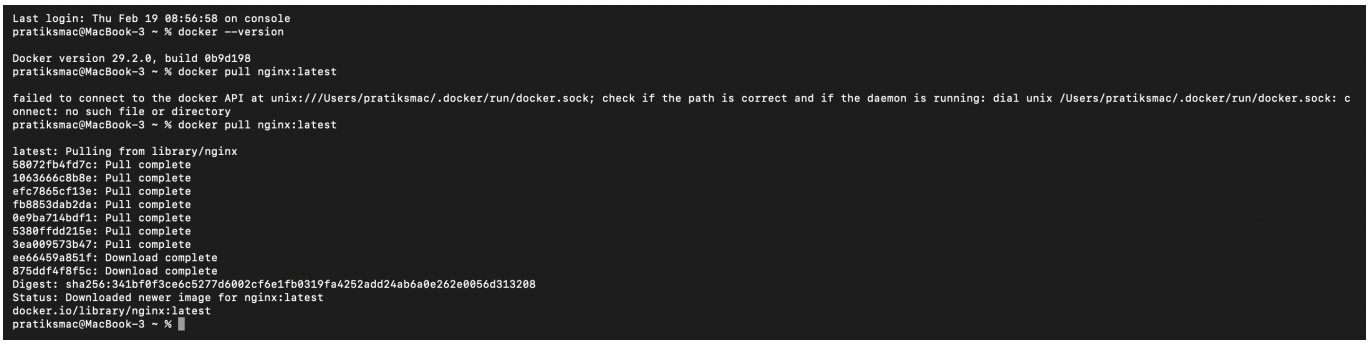
Images Used

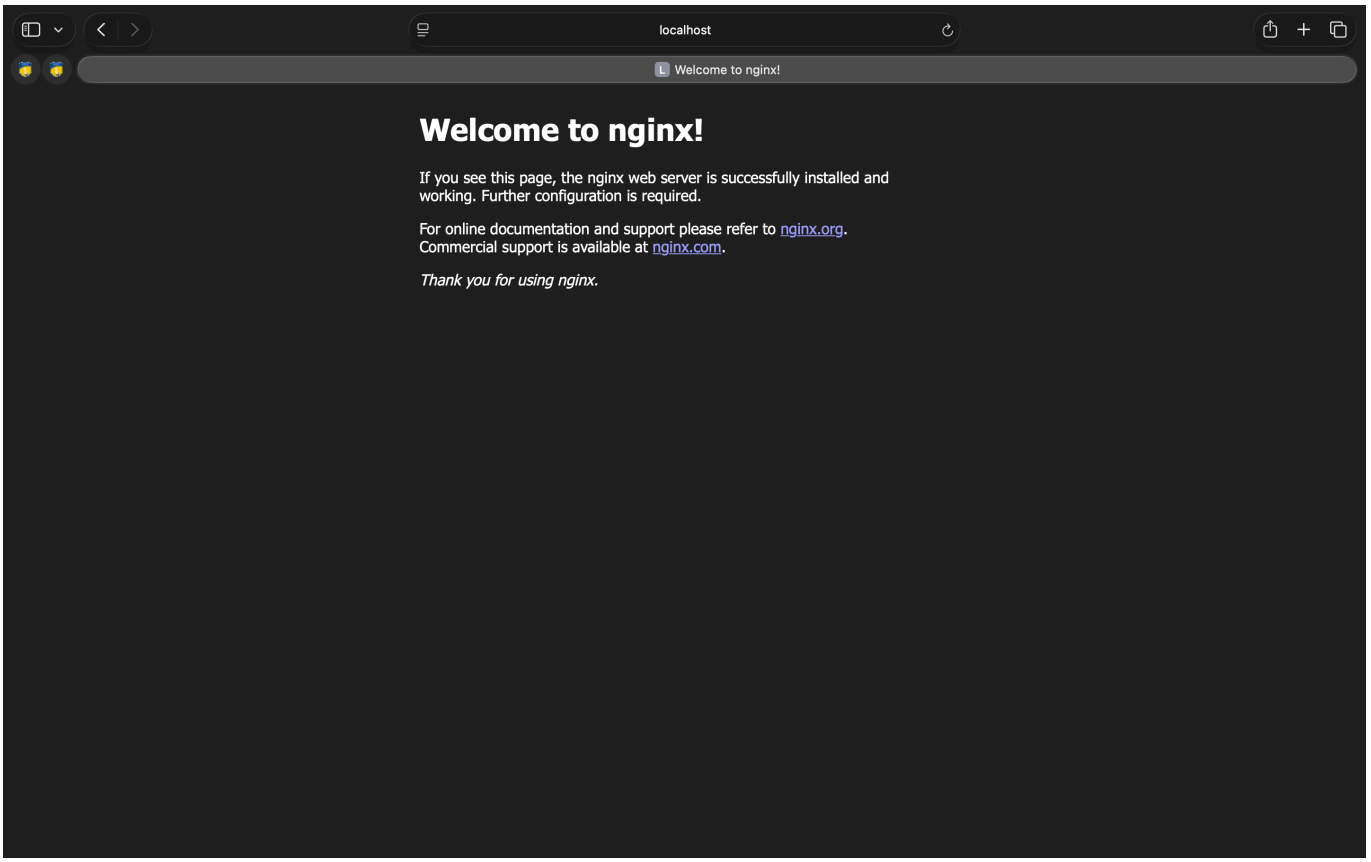
Image Name	Base OS	Purpose
`nginx`	Debian-based	Official default NGINX image
`nginx-ubuntu`	Ubuntu	Custom-built NGINX image using Ubuntu
`nginx-alpine`	Alpine Linux	Lightweight NGINX image

Running NGINX Containers

Run Default NGINX

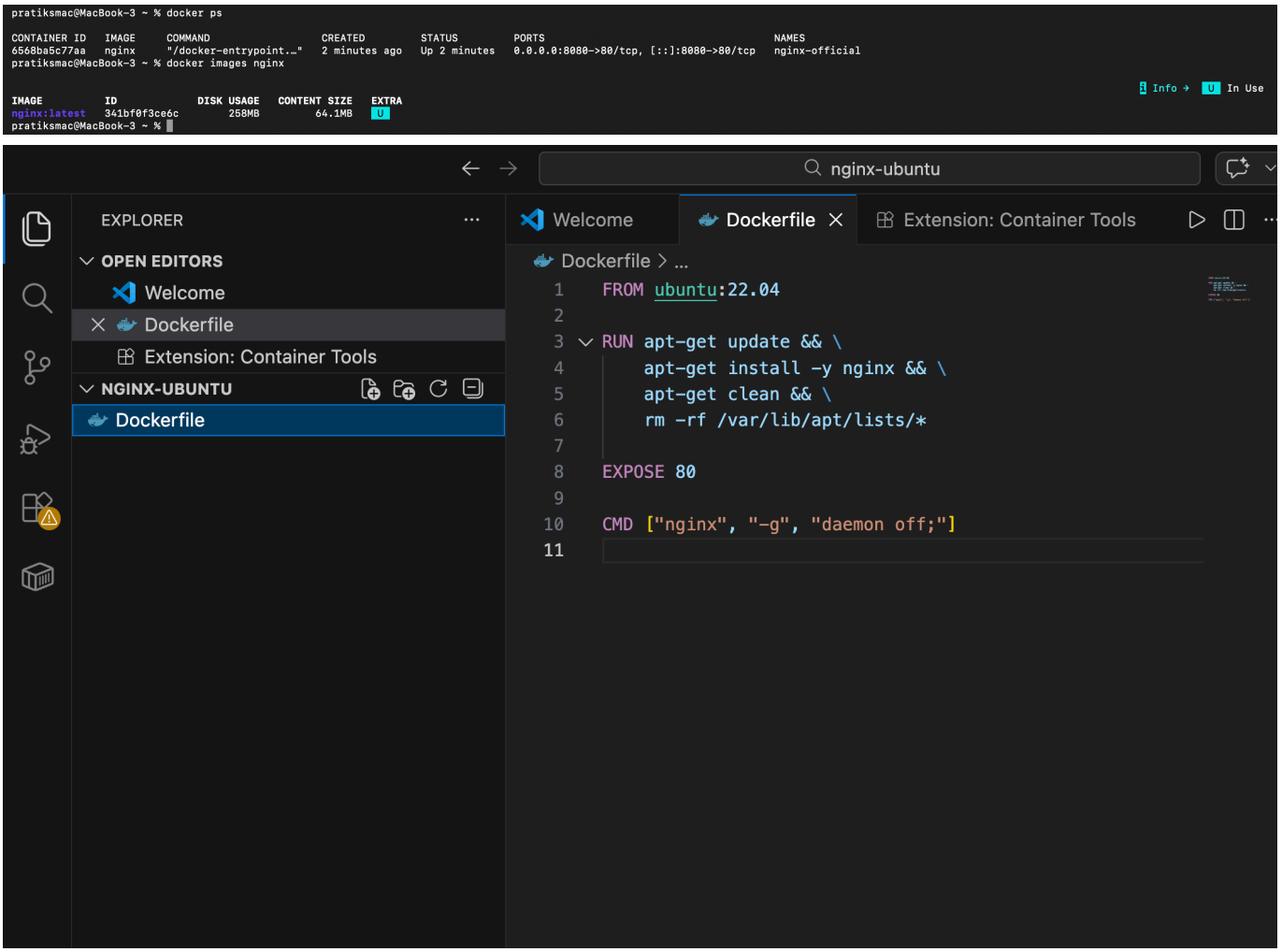
```
docker pull nginx:latest

docker run -d --name nginx-default -p 8080:80 nginx
```





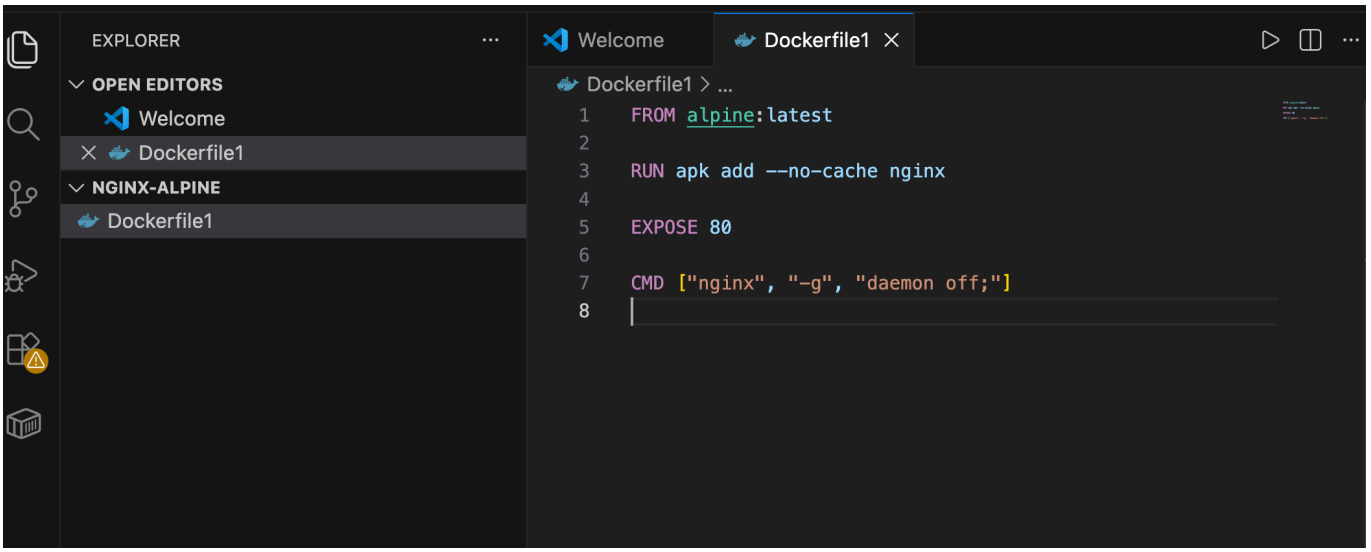
Run NGINX Built on Ubuntu

```
docker run -d --name nginx-ubuntu -p 8082:80 nginx-ubuntu
```



Run NGINX Alpine Version

```
docker run -d \
  --name nginx-alpine \
  -p 8087:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
```



Comparing Image Layers

To inspect how each image is built, we use:

```
docker history nginx
docker history nginx-ubuntu
docker history nginx:alpine
```

This command shows:

Number of layers Size of each layer Commands used to build each layer

Welcome to nginx!

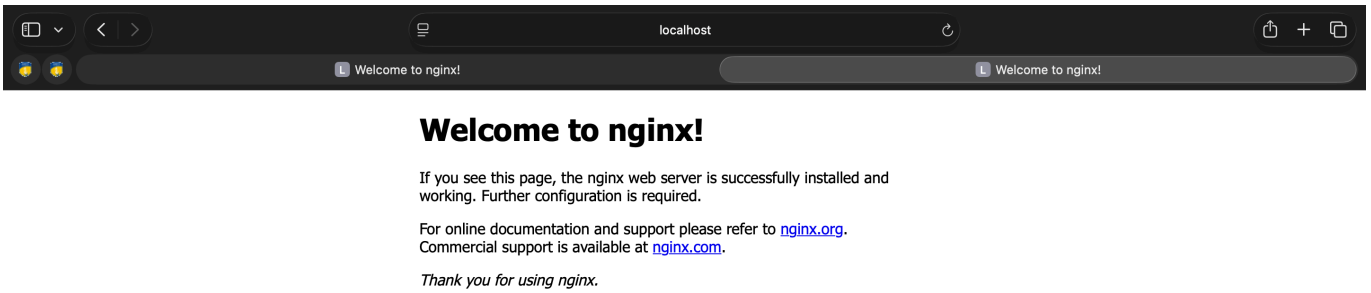
If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

```
[pratiksmac@MacBook-3 nginx-ubuntu % cd ..
[pratiksmac@MacBook-3 ~ % mkdir nginx-alpine
[pratiksmac@MacBook-3 ~ % cd nginx-alpine
 pratiksmac@MacBook-3 nginx-alpine % code .
```

```
pratiksmac@MacBook-3 nginx-alpine % docker build -t nginx-alpine .

[+] Building 0.0s (1/1) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 2B                                                                                                  0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
pratiksmac@MacBook-3 nginx-alpine % ls

Dockerfile
pratiksmac@MacBook-3 nginx-alpine % docker build -t nginx-alpine .

[+] Building 6.1s (6/6) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 133B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/alpine:latest                                                                    4.0s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 2B                                                                                                     0.0s
 => [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659              1.0s
 => => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659              0.0s
 => => sha256:d8ad8cd72600f46cc068e16c39046ebc76526e41051f43a8c249884b200936c0 4.20MB / 4.20MB                                      0.9s
 => => extracting sha256:d8ad8cd72600f46cc068e16c39046ebc76526e41051f43a8c249884b200936c0                                           0.1s
 => [2/2] RUN apk add --no-cache nginx                                                                                              0.9s
 => exporting to image                                                                                                              0.1s
 => => exporting layers                                                                                                             0.1s
 => => exporting manifest sha256:1dc92d1fc26221bc99594b5b3cc16d3dc8ba06640f90328bb9ebcf69560ae10d                                   0.0s
 => => exporting config sha256:18f1c5302f954ce35e5699636530a2965d253be5bd335a6eb41ebf8f777b70540                                   0.0s
 => => exporting attestation manifest sha256:079ed0e4548d645abfb1f01b4ce8a1fff17185e3514460de39562c574a11e13b                       0.0s
 => => exporting manifest list sha256:a6e330870ea23ab6c1c6701a96c3d8031208043 8c23e97b10e176f91c287d4a5                             0.0s
 => => naming to docker.io/library/nginx-alpine:latest                                                                              0.0s
 => => unpacking to docker.io/library/nginx-alpine:latest                                                                           0.0s
pratiksmac@MacBook-3 nginx-alpine %
```



Observations

Image	Size	Layers	Notes
`nginx`	Larger	More layers	Includes full Debian OS
`nginx-ubuntu`	Very Large	Many layers	Ubuntu base adds extra packages
`nginx:alpine`	Smallest	Fewer layers	Minimal OS, optimized for containers